Analysis of Skip Lists Compared to B+ Trees

Milad Abdi, Nathan Jacobson
Faculty of Applied Sciences
Simon Fraser University
Burnaby, Canada
mka321@sfu.ca, 301647901
nmj8@sfu.ca, 301639172
April 16, 2026

I. Abstract

Efficient data storage relies upon selecting optimal data structures, where Balanced Binary Search Trees (B+ Trees) have been the long standard for $O(\log n)$ operations. However, despite these performance guarantees, data structures like AVL or Red-Black Trees are notoriously difficult to properly implement and maintain due to their rebalancing logic and rigid structural constraints. In 1990, William Pugh introduced the Skip List as a probabilistic alternative, designed to mimic the efficiency of a B+ while having a much simpler implementation. Despite being a relatively new concept in the field of computer science, Skip Lists are widely used in high-performance real-world applications, such as the database Redis and the messaging platform Discord.

This paper provides a comparative analysis of the Skip List's performance when compared to an AVL Tree. Using custom implementations of an AVL Tree and a Skip List in C++, we measured the execution time of each data structure's standard operations: insertion, search, and deletion. Based on our benchmarks, Skip Lists perform comparably to AVL Trees, while being implemented using ~50% fewer lines of code and not needing rigorous testing, compared to the AVL Tree. Our findings conclude that Skip Lists are a viable alternative to B+ Trees, offering a superior ease of implementation and faster insertion/deletion operations while maintaining a search efficiency comparable to a traditional B+ Tree.

II. Background and Definitions

Abstract data type (ADT): A data model defined by what values it stores, and what operations it can perform. An ADT does not consider implementation details.

Database: An organized collection of data with efficient storage, as well as operations for retrieval, and modification of such data.

Sorted Set: An abstract data type similar to a hash where you can access items with a key, however every item has a score associated with it to have a set order for the items.

Binary Search Tree: A recursively defined Binary Tree (BT) where all nodes to the left of a subtree root are smaller, and all nodes to the right are bigger. When the tree is balanced, operations have logarithmic time complexity by using a binary search with logarithmic height. Binary Search Trees require balancing algorithms that maintain logarithmic height after many insertions. AVL, Red-black, Splay, and Fibonacci BSTs are all unique implementations of self-balancing trees denoted as B+ Trees [1].

Concurrency: Having different operations working simultaneously.

Exclusive Lock: An exclusive lock on an operation means all other queries which need to read or write to a locked operation must wait until the first operation is complete and unlocks.

III. Introduction

Skip Lists are a probabilistic data structure designed to be an alternative to B+ Trees [2]. B+ Trees are used extensively as the backbone to many common abstract data types (ADT) such as key-value stores and ordered lists [2]. B+ Trees are quite efficient for these ADTs; however their main drawback is the complexity required to implement the data structure and the balancing algorithms [2]. Skip Lists, on the other hand, are just a multi-layer linked list, meaning there is no need for a separate algorithm to balance the data structure on insertion. Even though these two data structures are implemented very differently, they still behave in a very similar way to each other [2]. Because of how simple implementing a Skip List is, they are used extensively to store data in various applications. For example, databases such as Redis and MongoDB use Skip

Lists to store sorted sets of data and improve database performance [3], [4]. Additionally, Skip Lists are also used to store data in large-scale, real-time applications such as Discord [5].

IV. History Behind Skip Lists

Over the years, there have been many data structures and ADTs introduced to the field of Computer Science. Skip Lists are a relatively modern idea, having only been first created in the early 1990s. Originally, they were designed to act like a B+ Tree, but with a simpler implementation [2]. However, it would take almost 20 years for Skip Lists to be implemented in real-world software, used by millions of people. The first publication of a skip list was in 1990 with William Pugh's research paper "Skip lists: a probabilistic alternative to balanced trees." This paper outlined how Skip Lists could be implemented as a simpler alternative to balanced trees [2]. Alongside this paper, Pugh also released an open source C library that contained an implementation of a Skip List [2]. The first well-known usage of Skip Lists was in 2009, when the in-memory database Redis launched. Redis used Skip Lists to implement their sorted sets, a hash where every element has an associated score which keeps the data ordered [4]. Later on in 2012, WiredTiger, another database, used Skip Lists to concurrently access the database across multiple threads simultaneously without needing to use locking mechanisms, such as mutexes [3]. This ensured that concurrent database connections wouldn't have to wait for other threads to finish their work with the database, thus greatly improving performance. In 2014, WiredTiger was acquired by MongoDB, a popular database platform, and was made the default underlying storage engine for MongoDB [6]. Finally, in 2019, Discord started to store every Discord server's member list using a sorted set, which is built on top of a Skip List [5]. Even though Skip

Lists are a relatively new creation, they are still used extensively for storing data in large scale applications and production-grade databases that handle millions of requests per day.

V. Major Applications

A. *MongoDB Concurrency*

As computer science has evolved, databases have come into the mainstream. They are used in nearly all real-world applications which require data storage, retrieval, and manipulation, such as social media, online shopping, banking, and healthcare systems [6]. The primary goal of any database is to support efficient data access. It is critical for data to be both read and written with as little delay as possible. Not only do slow databases irritate users, but in fields such as finance and healthcare, an inefficient database can become a massive burden.

The speed of data reading and writing is not just limited by its actual operation time. Since a database is queried many times at once, the more queries it can do simultaneously, the faster it will be. This is a key factor for MongoDB, who accordingly seek to maximize concurrency in their database [3]. The way to maximize concurrency is by "reducing how long an operation holds an exclusive lock" [3]. WiredTiger's solution was to reduce locking granularity, making lock management the bottleneck instead [3]. This comes from the idea that, "[t]he most effective way to reduce wait time for a lock is to reduce the size of what the lock is protecting" [7]. For example, instead of locking an entire table, we only lock its operation. The tradeoff, however, is the finer our granularity, the more overhead we need in order to manage our locks.

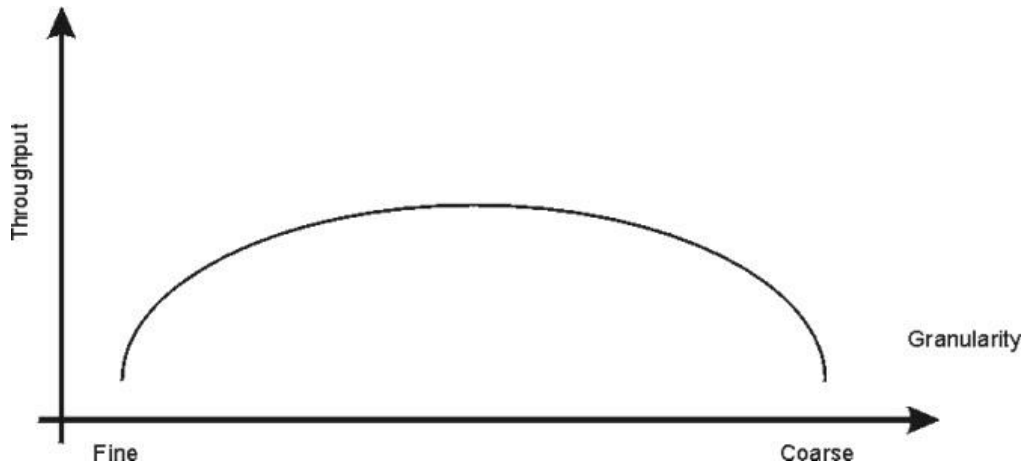


Fig. 1. Graph of throughput for varying levels of granularity, showing that the highest processing occurs at a granularity somewhere in the middle, rather than at either extreme [7].

Most lock-based databases have found the best balance for performance is locking at the page level [3]. If implemented, when performing an update operation, MongoDB would need to put an exclusive lock on the page until the update is finished. While this would be an improvement from locking the entire table, each write operation would still have to wait until the readers were finished and could not support concurrency with other writes for the same page. However, with Skip Lists they could achieve much better by removing the lock altogether. The lock-free Skip List works by making a new thread which constructs its operation to the data structure and then atomic-compare-and-swap (CAS) the change in place [3]. If the CAS is successful, the change was made. If another thread was changing the same page, then the CAS would fail, and the thread would retry.

As mentioned earlier, removing locks would increase our overhead. However, with lock-free Skip Lists, the complexity cost from threads retrying their update operations if another thread won the race is much less than using locks [3]. Moreover, the extra memory from the Skip

List is negligible as the probability of parallel lists exponentially decreases, making the space cost linear.

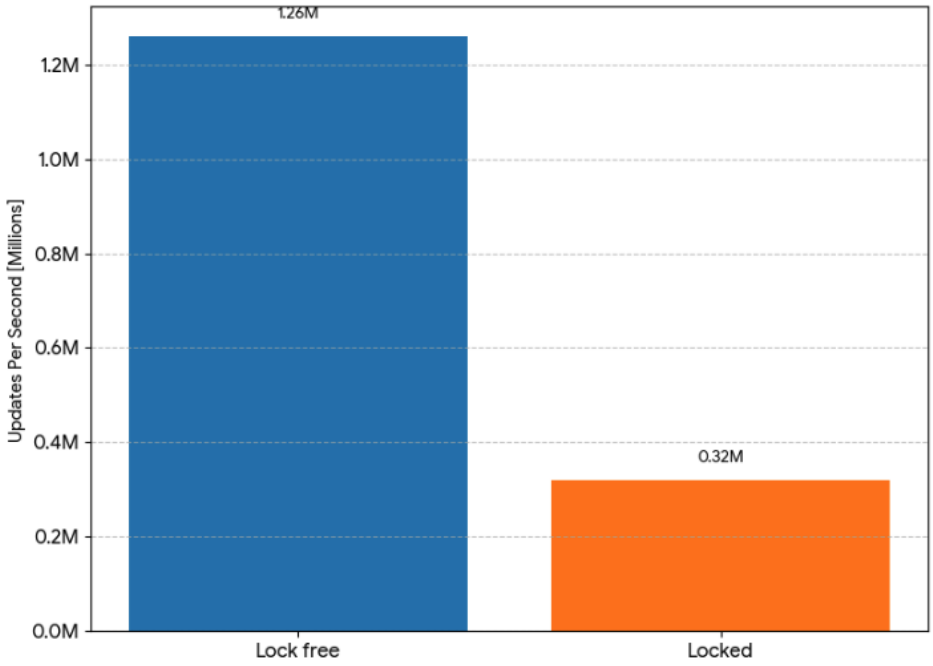


Fig. 2. MongoDB's comparison of the updating operation throughputs with their Lock free versus Locked implementations [3]. Using a Skip List for their lock free algorithm, they saw a performance boost of nearly 4x.

B. *Redis Sorted Set*

Redis (Remote Dictionary Server) is an open source, in-memory, NoSQL key/value store commonly used as a database, cache, message broker, and streaming engine [8], [9]. Being in-memory, Redis reads and writes extremely fast and has very low latency [8]. Alongside being fast, Redis supports many different data types for users to utilize such as Strings, Lists, and Sets [9].

Sorted Sets are a Redis data type which store a collection of unique elements ordered by a numerical score [4], [9]. They are implemented with a hash table for fast access and use a Skip

List to maintain its sorted order [4]. Being both fast and ordered, Sorted Sets are incredibly useful for leaderboards, job scheduling, and rate limiters [4], [9], [10].

The hash table portion of the Sorted Set provides constant look up time for the score of a member and the Skip List allows for logarithmic searching, inserts, and deletes [10]. Additionally, Sorted Sets have another common operation: determining the rank of an element in comparison to the others [4], [10]. We cannot store the rank with the element and its score because we would need to update all elements every insert/delete which would result in linear time [10]. Instead, for each element in the Skip List, we have a span variable which has the number of elements skipped before reaching the next node on its current Skip List level [10]. With this idea, we can get the rank in $O(\log(n) + m)$ time complexity where m is the range length [10].

C. Discord's Member List

Skip Lists are not just implemented in databases; they also have their place in large-scale applications for their scalability and performance. Among them is Discord, a gaming focused social media with text and call services between users — often through discord servers. Discord servers are communities where members can connect with each other. In 2023, Discord had roughly 200 million monthly users and 6.7 million active servers [11].

Discord has a sorted server member list in each server. However, as Discord grew, a new problem arrived for them with it. Servers began having member lists as large as 100,000. Thus, when a new member would join, they would need to rebuild a new list. Requiring a fast but ordered list structure, Discord benchmarked various data structures such as Lists and Map Sets,

but found them too slow for the task [5]. Ordered Sets performed better but were still slow under scale.

Discord's solution was to utilize Skip Lists in a unique way that made their ordered sets efficient at scale. Originally, Discord used OrdSets (ordered sets) that were included with the Elixir programming language's standard library to store server member lists. However, the performance when dealing with lots of elements was too slow to handle the amount of traffic that Discord received daily [5]. Discord was able to address this issue by using a Skip List; they broke up their OrdSets into many smaller collections of items and then used a Skip List to allow for quick searching of items by having the OrdSets as the bottom layer of the Skip List. This produced a nearly 5x performance improvement when compared to using a singular OrdSet [5]. By deploying their Sorted List, Discord has seen a vast performance boost with no measurable hit to their memory [5].

VI. Implementation

To compare Skip Lists with B+ Trees, we benchmarked the three core operations of insertion, deletion, and searching. Because these data structures are ordered, we only implemented them with their keys. This is because dealing with a hashing algorithm and data values would be unnecessary, as the extra time would be mutually added. There is, however, more to be said about what kinds of keys these different data structures prefer that our tests dived into. You can view our C++ implementation/benchmarks here.

A. Skip List Implementation

One of the major selling points of Skip Lists is how easy they are to implement compared to AVL Trees and Red-Black Trees. To insert data into a Skip List, we search every layer (from

the top) for a node that is greater than the item we want to insert. If we don't find a node that satisfies that condition, we jump down to the next layer. Otherwise, if we do, insert the new node into the current layer. After the node has been inserted into the current layer, we then do a "coin flip" (50/50 random number generation) to determine whether we should add it to the layers above our current layer. If the "coin flip" is "heads", we insert into the layer above and move up another layer but if it's "tails" we stop inserting.

Deletion is a different but still simple process. Start in the highest layer and traverse until you find a node that is greater or equal to your target value. If a node is found to satisfy that condition, store it a temporary array with the index being the current layer count. Drop down one level and repeat until the base level is reached. After this search, the first element in the temporary array should be the value we want to delete and thus we can delete it. However, if it is not the element we want to delete, the item does not exist in the list, and we can terminate this operation. The final step of deletion is to go back through all the nodes stored in the temporary array and update their pointers, cutting out the deleted node.

Finally, searching through a Skip List is extremely straightforward and is essentially a binary search through a linked list, making searches very efficient.

```
Level 3:  [HEAD] ------------------------> [30] ------------------> [NULL]
              |                              |
Level 2:  [HEAD] --------> [10] ---------> [30] ---------> [50] --> [NULL]
              |              |               |               |
Level 1:  [HEAD] -> [5] -> [10] -> [20] -> [30] -> [40] -> [50] ->  [NULL]
```

Fig. 3. A visual representation of a Skip List [3].

Using Fig. 3 (and setting our target value to 50), searching in a Skip List works as follows: we would start at level 3 and see the next node is 30, which is less than 50, so we need to move down a layer. In level 2, we see the next node is 50 which is the value we are looking for and thus we have completed the search. Searching through a Skip List and a Balanced Tree should result in the same logarithmic time complexity. However, in a real-world system, B+ Trees tend to be faster due to them guaranteeing only two pointers per node, whereas Skip Lists can have more potentially leading to less efficient comparison operations.

B. AVL Implementation

Unlike Skip Lists, AVL Tree operations require heavy recursion and many extra steps. To insert into the AVL Tree, we recursively go down the tree in logarithmic time using the BST ordering property. Once inserted, we go back up the recursion stack, updating the node heights, and fixing any newly unbalanced subtrees with rotations.

Deletion is very similar except with one additional complication. We need to use the BST deleting algorithm and handle the three cases of deleting a node with no child, one child, and two children. Deleting a node with no children is as easy as deleting it. With one child, we need to update the parent to point to the deleted node's child. Finally, with two children, we must find the successor of the node to delete, which is the next node in order. This turns out to simply be the smallest element from the right child. We then swap the successor and the node to delete and continue our deletion algorithm to the successor as it is guaranteed to have at most one child.

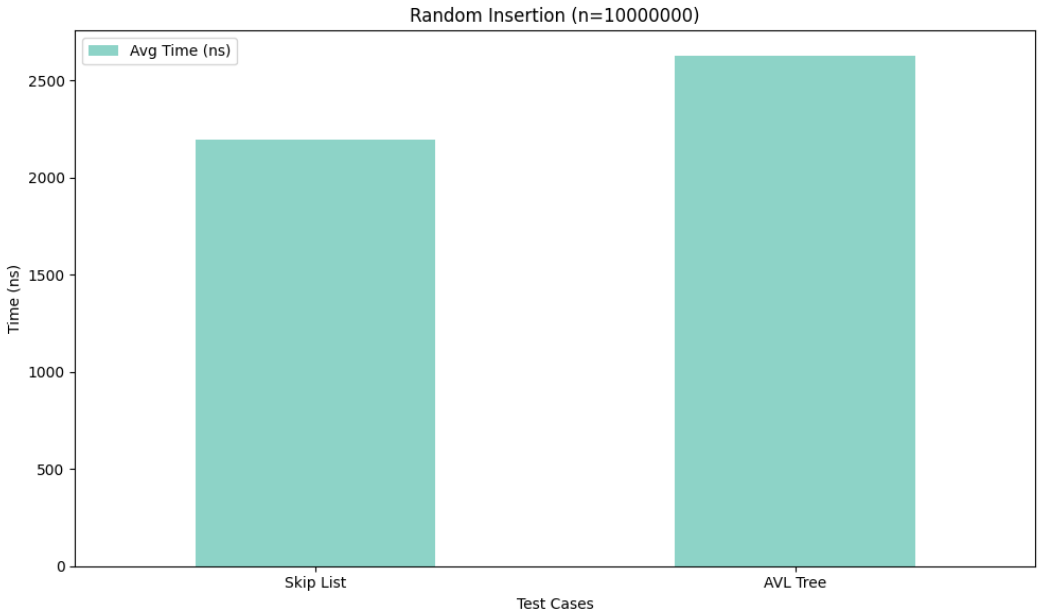
Finally, the big payoff from the added complexities from insertion and deletion come into play with searching. From our extra work, we have a balanced and ordered binary tree which can

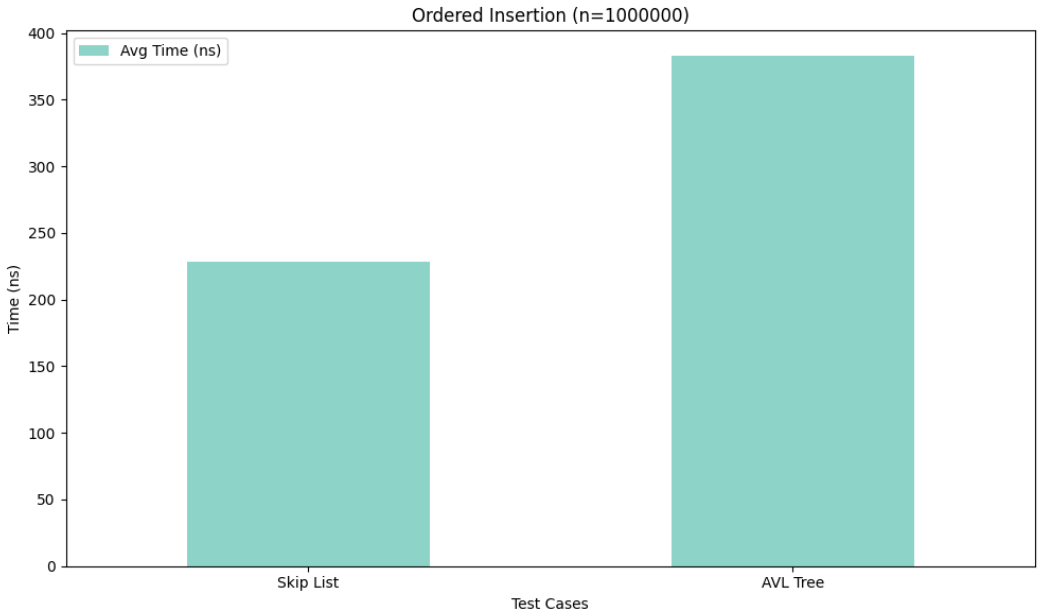
be very easily searched in logarithmic time by traversing the tree like a binary search and going to the left subtree if the key is smaller than the current node, and right if it's larger.

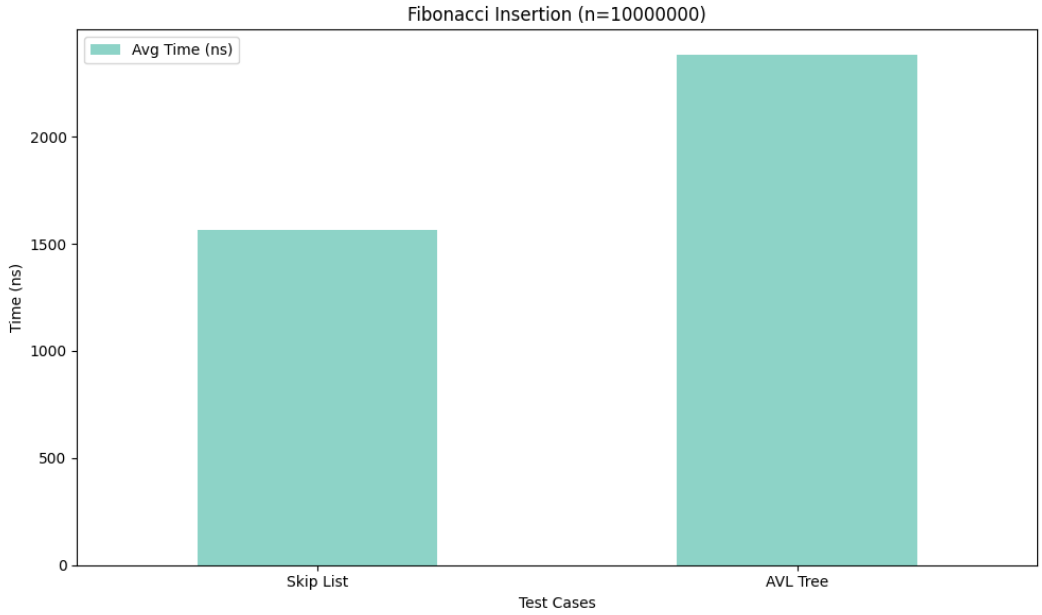
VII. Experiments and Evaluation

A. Insertion

We benchmarked three different insertion tests. The Skip List was on average ~30% faster across all three. Random insertion was the closest between the two, with the AVL Tree only trailing behind slightly. However, with the ordered and Fibonacci insertion cases, Skip List preformed nearly twice as fast. These differences likely arise from how the AVL Tree maintains its balance, resulting in different patterns of insertions affecting the number of extra operations the AVL will need to do.

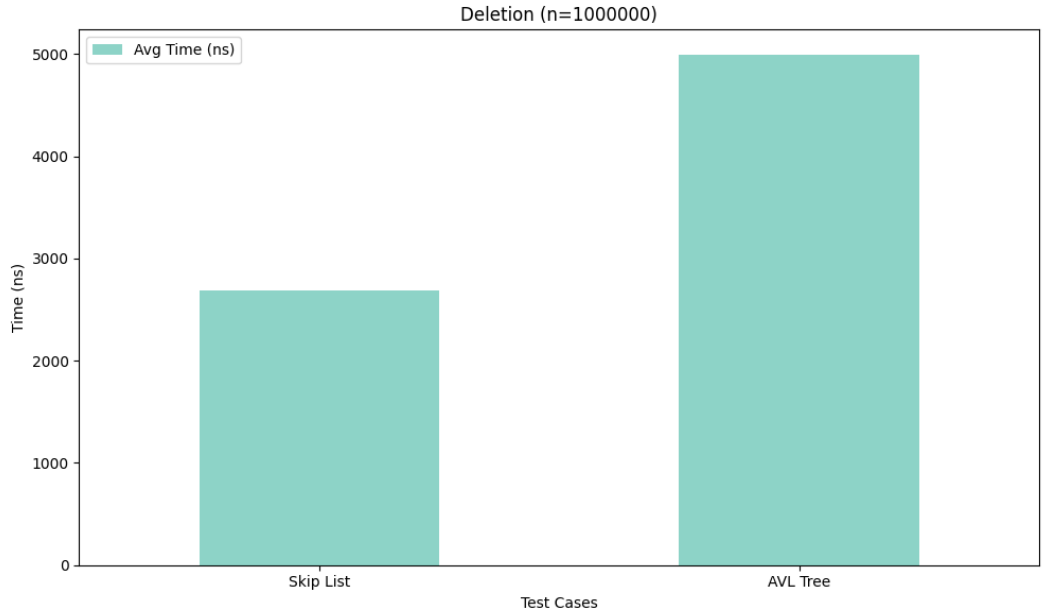



Ordered Insertion (n=1000000)
Avg Time (ns)
Time (ns)
400
350
300
250
200
150
100
50
0
Skip List
AVL Tree
Test Cases


Fibonacci Insertion (n=10000000)
Avg Time (ns)
Time (ns)
2000
1500
1000
500
0
Skip List
AVL Tree
Test Cases

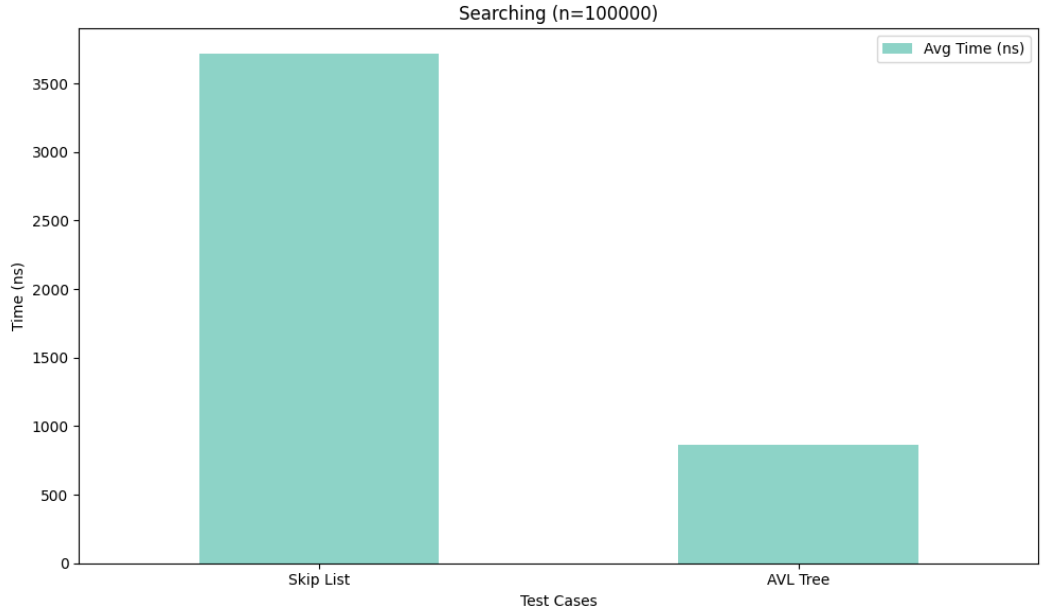
B. Deletion

For deletion, Skip Lists are again about twice as fast as AVL Trees due to not requiring any additional structural operations to keep their fast operations.



C. Searching

Lastly, we did see a noticeable decrease in performance in searching for the Skip Lists. AVL Trees guarantee a balanced ordered tree, making searches extremely fast and reliable. This results in it searching about three times faster than Skip Lists.



VIII. Discussion

Some limitations in our benchmarking includes our lack of hardware, making our results fixed to small data, potentially missing wider implications from large datasets. Moreover, we faced challenges with our G++ compiler. When we tested our benchmarks with optimizations on, we saw a significant performance increase in AVL Trees compared to Skip Lists.

Nevertheless, we see a clear picture of the landscape between Skip Lists and B+ Trees. Skip Lists are faster at inserting and deleting because they require less overhead, not needing to maintain a rigid structure. The cost is that Skip Lists have a much worse upper bound performance [2]. A Skip List has a worse-case time complexity of $O(n)$ for every operation (search, insert, remove) whereas a B+ Tree has a worse-case time complexity of $O(\log n)$ for every operation [2], [12]. Although, Skip Lists have a much larger worse-case time complexity, it is extremely rare for a Skip List to encounter data that causes it to perform at its worse-case time complexity [2]. That said, its searches are still slower than the extremely consistent B+ Trees due to the B+ Tree's rigid balanced structure.

Another limitation of a Skip List is the fact that Skip Lists require more comparisons to be run for its operations [2]. If you use floating-point numbers or strings as keys in the Skip List, this will create a much larger performance impact due to the more complicated nature of making comparisons with these data types. For example, comparing strings requires CPUs to loop through the entirety of the two strings every comparison. However, if you only use data types that CPUs can easily compare (such as integers), the performance impact will be negligible. An integer only requires a single CPU cycle to run a comparison; when many comparisons need to be made, having a constant time complexity (versus a linear time complexity) will greatly

improve performance. As long as the key type of the Skip List can be easily compared, it will have performance comparable to a B+ Tree [2].

In the future, we would be interested in benchmarking Skip Lists against more B+ Trees, having better hardware for a larger and more complex benchmark, and further exploring how these data structures compare in real-world systems.

IX. Conclusion

Skip Lists are a viable alternative to B+ Trees, being simpler to implement while achieving the same goal with similar performance. Being first introduced in 1990 by William Pugh's "Skip lists: a probabilistic alternative to balanced trees", they are a very new concept in the field of computing science. Even as Skip Lists are a relatively young data structure, they still are widely used in real-world applications such as databases and messaging platforms. Redis uses Skip Lists to implement Sorted Sets in their key-value database while MongoDB uses Skip Lists to achieve a lock-free way to concurrently write to a database. Additionally, popular messaging platform Discord uses Skip Lists to store the member lists of servers (large group chats).

We created a C++ implementation of a Skip List and AVL Tree to benchmark and compare the performance of their standard operations (insertion, search, deletion). Based on our C++ implementation and benchmarks, Skip Lists are on average ~30% faster at inserting data and ~46% faster at removing data, however AVL Trees are still ~76% faster at searching. Our C++ implementation of a Skip List required ~50% fewer lines of code compared to the AVL Tree, significantly reducing the surface area for logic errors to occur. Our conclusion is that Skip

Lists are indeed a highly viable alternative to a B+ Tree, offering faster modifications (insertion/deletion), slower searching, and a much simpler implementation when compared to traditional B+ Trees.

References

[1] S. Roura, “Fibonacci BSTs: A new balancing method for binary search trees,” *Theor. Comput. Sci.*, vol. 482, pp. 48–59, Apr. 2013, doi: 10.1016/j.tcs.2012.11.027.

[2] W. Pugh, “Skip lists: a probabilistic alternative to balanced trees,” *Commun ACM*, vol. 33, no. 6, pp. 668–676, Jun. 1990, doi: 10.1145/78973.78977.

[3] A. Gorrod, “To Lock or Not: MongoDB’s Lock-Free B-Tree Unlocks Throughput,” MongoDB. Accessed: Mar. 28, 2026. [Online]. Available: https://www.mongodb.com/company/blog/engineering/to-lock-or-not-mongodbs-lock-free-b-tree-unlocks-throughput

[4] Redis, “Redis Sorted Sets,” Redis. Accessed: Mar. 28, 2026. [Online]. Available: https://redis.io/glossary/redis-sorted-sets/

[5] M. Nowack, “Using Rust to Scale Elixir for 11 Million Concurrent Users.” Accessed: Mar. 29, 2026. [Online]. Available: https://discord.com/blog/using-rust-to-scale-elixir-for-11-million-concurrent-users

[6] “MongoDB Acquires WiredTiger Inc.,” MongoDB. Accessed: Mar. 30, 2026. [Online]. Available: https://www.mongodb.com/company/newsroom/press-releases/wired-tiger

[7] IBM, “AIX.” Accessed: Apr. 01, 2026. [Online]. Available: https://www.ibm.com/docs/en/aix/7.2.0?topic=locks-lock-granularity

[8] IBM, “What is Redis Explained?” Accessed: Mar. 28, 2026. [Online]. Available: https://www.ibm.com/think/topics/redis

[9] J. William, “What is Redis?: An Overview,” Redis. Accessed: Apr. 02, 2026. [Online]. Available: https://redis.io/tutorials/what-is-redis/

[10] Jothi, “Redis Sorted Sets - Under the Hood,” Learn By Doing. Accessed: Apr. 04, 2026. [Online]. Available: https://jothipn.github.io/2023/04/07/redis-sorted-set.html

[11] D. Curry, “Discord Revenue and Usage Statistics (2026),” Business of Apps. Accessed: Apr. 04, 2026. [Online]. Available: https://www.businessofapps.com/data/discord-statistics/

[12] A. Mozafari, “Search Trees,” Simon Fraser University, Burnaby, BC, Apr. 01, 2026.